0ARQUITECTURA DE COMPUTADORES
BOLETÍN TEMA 3: ENTRADA/SALIDA

Analíticos

Ejercicio 1. Un periférico genera una interrupción cuando está preparado para realizar una operación. Además, dispone de un registro de estado cuyo bit menos significativo toma el valor 1 cuando está listo. A continuación, se muestra la RTI del periférico. Explique muy brevemente el error que contiene y su posible solución.

```
void rti (){
    ...
    while ((inb(STATUS_PORT_X) & 1)== 0);
    operacion_ES();
}
```

Ejercicio 2. Un computador con un procesador secuencial está conectado a un dispositivo de E/S que consta de dos sensores: se transfiere el valor tomado por cada sensor en cada petición de lectura. Teniendo en cuenta que:

- El dispositivo de E/S necesita un promedio de 109ns para procesar la petición.
- El envío de cada petición tarda 4ns y la transferencia del dato 2ns.
- El cambio de contexto necesita 10ns.
- La emisión de la señal de interrupción es inmediata.
- Suponiendo que se van a leer ambos sensores (por lo que requiere dos transferencias), calcule el porcentaje de tiempo que la CPU dedica a realizar las dos transferencias:
  - **a)** Utilizando E/S programada.
  - **b)** Utilizando E/S por Interrupciones considerando que la segunda petición de lectura se realiza dentro de la rutina de tratamiento de interrupción. Muestre los cálculos y describa su planteamiento ayudándose de una línea de tiempo.

Ejercicio 3. Un procesador de 1GHz de reloj con un CPI medio de 10 en el que cada operación de E/S requiere 10 instrucciones de CPU, está conectada a una impresora. La impresora imprime a una velocidad de 80 caracteres/segundo y no posee memoria interna. Calcule el tiempo total para imprimir 100 caracteres. Supondremos terminada la transferencia cuando se envíe el último carácter (sin esperar a que la impresora lo procese). Nota: el controlador de E/S posee un registro de datos de 8 bits y una velocidad de transferencia de 100Kb/segundo.

- **a)** Suponiendo que se utiliza E/S por polling con un bucle para la espera activa de 20 instrucciones. La espera activa es necesaria para esperar a que la impresora haya procesado el último carácter enviado. Suponga que, en código de la espera activa, la lectura del registro de estado correspondiente se realiza en la primera instrucción.
- **b)** Mediante interrupciones. Considerando que el tiempo desde que se activa la interrupción hasta que empieza la ejecución de la rutina de servicio de interrupción es de 500 ciclos.

Ejercicio 4. Tenemos un escáner que transmite datos a razón de 8000 Bytes/s. Se pide comparar qué método (DMA o interrupciones) sería más adecuado, es decir, consumiría menos ciclos del procesador sabiendo que:

- Por interrupciones, el tiempo gastado en cada transferencia de 4 Bytes es 15 ciclos, el procesador tarda un tiempo medio de 1 ciclo en cambiar de contexto y 4 ciclos en configurar una transferencia.
- Por DMA, iniciar cada transferencia de 1000 B cuesta 200 ciclos y el tratamiento de la interrupción al terminar la operación completa cuesta otros 150 ciclos. Además, el procesador dedica 175 ciclos en configurar el controlador de DMA y 75 ciclos al cambiar de contexto tras finalizar la operación.
- Indique qué mecanismo es más rápido y qué aceleración tiene respecto al otro.

Ejercicio 5. Sea un computador que se comunica con un dispositivo mediante interrupciones.

- El dispositivo actualiza su registro de datos continuamente con un nuevo dato cada **55** ciclos de reloj (independientemente de si el valor es leído o no), provocando una interrupción en cada actualización.
- Respecto al computador, sabemos que:
  - Su procesador es secuencial y ejecuta una **instrucción cada 5 ciclos.**
  - Transferir el dato desde el registro de datos del dispositivo requiere 3 instrucciones, denominadas **T1 y T2, T3**.
  - La configuración del computador y del dispositivo requiere 4 instrucciones en total, denominadas **C1**, **C2**, **C3** y **C4**.
  - El cambio de contexto requiere 10 ciclos de reloj (utilice **XT** por cada 5 ciclos para equipararlo a una instrucción)

- Tras la configuración del dispositivo, el procesador comienza a ejecutar un bucle de 3 instrucciones denominadas **I1**, **I2** e **I3** cuya ejecución no está relacionada con la comunicación que realiza con el dispositivo.

La siguiente tabla representa una línea de tiempo (de izquierda a derecha) en la que cada celda corresponde a un intervalo de tiempo de 5 ciclos. Se pide que especifique sobre la línea de tiempo las instrucciones o tareas en periodos de 5 ciclos (utilizando las denominaciones descritas: A1..A3, C1..C4, etc.) que realiza la CPU al transferir datos continuamente desde el dispositivo respetando la temporización..

Ciclo	5	10	15	20	25	30	35	40	45	50	55	60	65	70	75	80	85	90	95	100	…
Secuencia	C1	C2																			…

…	105	110	115	120	125	130	135	140	145	150	155	160	165	170	175	180	185	190	195	200
…																				

E/S mapeada (ensamblador RISC-V)

Ejercicio 6. Dispone del siguiente código en el que se realiza un acceso a un dispositivo de E/S.

```
   addi x10,x0,0x100
a: lbu x12,0(x10)
   andi x13,x12,0x40
   bne x13,x0,a
   lbu x20,2(x10)
   slli x20,x20,6
   lbu x15,1(x10)
   andi x15,x15,0x3F
   or x15,x15,x20
   andi x15,x15,0xDF
   sb x15,1(x10)
```

a) Describa el dispositivo al que se refiere y su funcionamiento. Para ello, utilice el recuadro de más abajo para detallar los registros internos que posee: dirección de cada registro, tipo de registro y los bits involucrados en la comunicación con una letra distintiva (y única) que lo identifique, deje en blanco el recuadro de los bits que no son de interés en el dispositivo. Se trata de que describa el comportamiento del dispositivo a través del código (no que indique el resultado final tras la ejecución).

Para determinar el funcionamiento del dispositivo, describa cómo actúan los bits identificados, indicando la letra identificativa y su comportamiento (Ej1: Cuando el bit X está '1'… Ej2: Para enviar un dato hay que…)

7	6	5	4	3	2	1	0	Tipo de registro y Dirección
								REG. _____ DIR 0x_____
								REG. _____ DIR 0x_____
								REG. _____ DIR 0x_____

b) Realice una versión del programa anterior en C mediante E/S separada por polling.

c) Modifique el programa del enunciado para adecuarlo a un dispositivo idéntico cuyas señales de estado y control utilizan lógica inversa (las líneas activas en alta ahora son en baja y viceversa), las señales asociadas a datos deben mantenerse intactas. Indique solo las instrucciones que cambia en la zona habilitada para ello.

Ejercicio 7. Sea un **RISC-V** que se conecta a un dispositivo capaz de medir la temperatura y la humedad. Este dispositivo cuenta con los siguientes registros:

	7	**6**	**5**	**4**	**3**	**2**	**1**	**0**
Reg. de control y estado (0xAC)	X	**Leer**	X	**TH**	X	**Disp***	X	X

- **Leer**: Solicita la lectura en el sensor. El bit TH especificará qué magnitud debe leer.
- **TH:** Especifica si la petición es para leer la temperatura (0) o la humedad (1):
- **Disp*:** Indica si ya está disponible en el registro de datos la magnitud solicitada. **0** → Magnitud disponible.

Reg. de datos (0xAD): Registro de **16 bits** que guarda el valor solicitado. La temperatura puede ser negativa.

Realice un programa en **ensamblador** que guarde en ***x5*** la temperatura actual. **Incluya un comentario por línea**.

```
.text
     li x10,0xAC
     …
```

Ejercicio 8. Dispone de un procesador RISC-V con E/S mapeada en memoria y los tres siguientes registros de E/S:

	7	***6***	***5***	***4***	***3***	***2***	***1***	***0***
Reg A (0x53)	*0*	*1*	*1*	*1*	*0*	*1*	*0*	*0*
Reg B (0x54)	*1*	*1*	*0*	*0*	*0*	*0*	*0*	*1*
Reg C (0x55)	*0*	*0*	*0*	*1*	*1*	*X*	*1*	*1*

Partiendo del siguiente código de carga de direcciones:

```
.text
   li x10, 0x53   # Dirección de Reg A
   li x11, 0x54   # Dirección de Reg B
   li x12, 0x55   # Dirección de Reg C
```

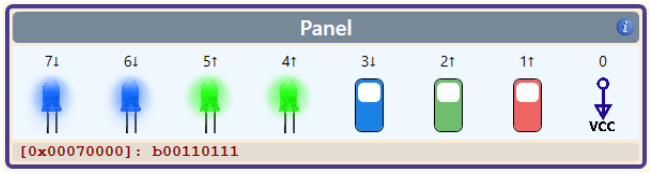
a) Dado el siguiente trozo de código, indique (sobre el contenido de los registros anteriores), qué cambio habría.

```
lbu x20, 0(x10)
andi x20, x20, 0x10
srli x20, x20, 3
lbu x21, 0(x11)
or x21, x21, x20
sbu x21, 0(x11)
```

b) Modifique el código anterior para que el cambio realizado afecte al bit 6 del registro C.

c) Escriba un código en ensamblador de RISC-V para que conmute el valor del bit 2 del registro C: si es '0' debe pasar a valer '1'; y si es '1' debe pasar a valer '0'. Inicialmente, desconocemos su contenido (marcado como 'X' en el enunciado).

Ejercicio 9. Realice el código ensamblador que se solicita en cada apartado para un procesador RISC-V con E/S mapeada en memoria conectado a un dispositivo consistente en un panel con tres interruptores y cuatro leds. La flecha hacia arriba indica lógica positiva y hacia abajo lógica negativa. Por cada instrucción añada un breve comentario que describa la funcionalidad.



Suponiendo:

```
.text
   li x6, 0x60000
```

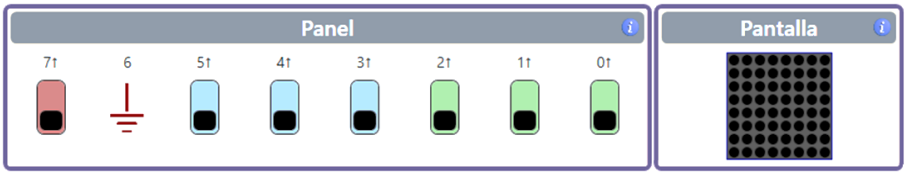
a) Espera activa mientras el interruptor conectado al pin 1 está desactivo**.**

b) Si el interruptor conectado al pin 2 está activo, apague los leds conectados a los pines 4 y 5.

c) Si interruptor conectado al pin 3 está activo, apague los leds conectados a los pines 6 y 7.

d) Encienda todos los leds.

Ejercicio 10. Se quiere implementar en ensamblador un juguete didáctico formado por un RISC-V conectado a dos dispositivos: un panel con 7 interruptores y una pantalla de 8 x 8 pixeles.



Panel (0xFFFFF) – Dispositivo de entrada:

- Bit 7: activo en alta. Borrado de la pantalla.
- Bits 5-3: identificador de columna en binario.
- Bits 2-0: identificador de fila en binario.

Pantalla – Dispositivo de salida (formado por tres registros):

- Registro de Datos 1 (0x100000): configurar fila de la pantalla.
- Registro de Datos 2 (0x100001): configurar columna de la pantalla.
- Registro de Estado/Control (0x100002). Formado por:

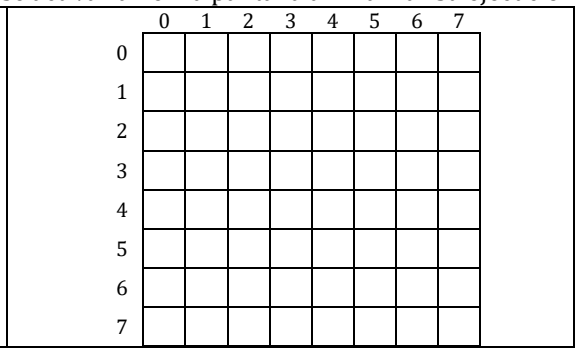
7	6	5	4	3	2	1	0
	Clear		Set*		Busy		

- Busy: activo en alta. Indica si está ocupado.
- Clear: activo en alta. Borra la pantalla.
- Set*: activo en baja. Activa el pixel especificado en los Registros de Datos 1 (fila) y 2 (columna).

Suponiendo que el registro '*x6*' posee el valor *0xFFFFF* y el registro '*x7*' el valor *0x100000*, se pide lo siguiente:

a) Con el siguiente código, dibuje qué píxeles se activarían en la pantalla al finalizar su ejecución.

```
        addi x10, x0, 0
        addi x11, x0, 8
rep:    lbu x4, 2(x7)
        andi x4, x4, 0x04
        bne x4, x0, rep
        sb x10, 0(x7)
        sb x10, 1(x7)
        lbu x4, 2(x7)
        andi x4, x4, 0xEF
        sb x4, 2(x7)
        addi x10, x10, 1
        bne x10, x11, rep
```



b) Implemente el borrado de la pantalla. Para ello, consulte si está activo el bit correspondiente del panel y, en ese caso, borre la pantalla (teniendo en cuenta que no puede hacerlo si está ocupada).

c) Implemente el código que lea la fila y columna del panel para, posteriormente, encenderlo en la pantalla.

E/S separada (programación en C)

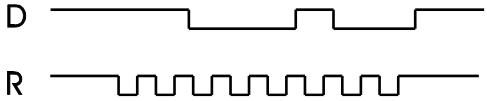
Ejercicio 11. Para un procesador que posee un sistema de E/S separada, escriba el código C que realice las tareas indicadas en cada apartado para el siguiente registro:

	7	**6**	**5**	**4**	**3**	**2**	**1**	**0**
Dir 0x0F	A (2 bits)		B	C	D	E	F	G

a) Modifique C a '1'.
b) Modifique E a '0'.
c) Consulte el valor de B. En caso de ser '0', escriba un '1' en G y, en caso de ser '1', escriba un '0' en F.
d) Ponga G a '1' si C vale '1' y F vale '0'.
e) Espere a que D valga '0' y, tras ello, copie el contenido de C a E.
f) Escriba el valor de la variable "unsigned char c" (que solo puede abarcar valores de 0 a 3), en A.

Ejercicio 12. Suponga un dispositivo E/S, con un único puerto de entrada de 8 bits, con el bit más significativo (MSB) normalmente a 1. Cuando baja a 0, indica que ha llegado un nuevo dato por los 7 bits restantes. Al leerlos, automáticamente el MSB vuelve a 1. Escriba una función que espere un dato (de 8 bits) y lo devuelva en una variable char.

Ejercicio 13. Se quiere construir un sistema de comunicación serie síncrona entre dos dispositivos, usando dos líneas: R y D. Los datos se reciben bit a bit por la línea D, con un nuevo bit por cada flanco de bajada del reloj R, y comenzando por el más significativo. Si no se reciben datos, la línea de reloj permanece a 1. Por ejemplo, el dato 11000100 (en binario):



Esta información es mandada por el dispositivo origen, mientras que el dispositivo destino se encarga de controlar los valores del reloj e ir leyendo de la línea de datos cuando sea oportuno. El receptor, dispone de un controlador de E/S que recoge el valor de estas dos señales de manera constante y lo guarda en un registro de datos (al guardar un nuevo valor, sustituye el valor anterior del registro anteriormente mencionado), al que se puede acceder mediante una lectura a la dirección 0x1000. En este registro de 8 bits, el bit 0 se encuentra conectado a la señal de reloj (R) y el bit 1 a la señal de datos (D).

Además, tiene un registro de control (accesible en la dirección 0x1001) mediante el que se controla la activación del dispositivo: El bit 0 habilita el dispositivo si está a uno, y el bit 1, habilita la generación de interrupciones). Si el dispositivo genera interrupciones, producirá una interrupción al procesador cuyo vector de interrupción es el 27).

a) Diseñe un programa que inicialice el periférico para la adquisición de datos por E/S por polling. El programa ha de recoger 1024 bytes y después, desactivar el dispositivo. Recuerde que, por cada byte, tendrá que hacer 8 lecturas de la señal D (una por bit), esperando los flancos de bajada de la señal R.

Además, tendrá que concatenar estos bits mediante desplazamientos para obtener un byte. Cada vez que forme un byte, deberá almacenarlo en un vector de 1024 posiciones (unsigned char v[1024];).

b) Realice el apartado anterior usando interrupciones. Para ello, la interrupción se disparará ante un flanco de bajada de la señal R. Deberá hacer una única lectura de D y controlar el número de bits (de los 8 del byte) que lleva leídos, así como el total de 1024 bytes límite a leer.

Ejercicio 14. Sea un dispositivo para el control de nivel de agua que consta de 4 bombas de agua y 4 sensores: sensor inteligente que cambia a 1 cuando está lleno y cambia a 0 cuando está vació. Cada sensor y cada bomba está instalado en un depósito, de forma que se pueden controlar 4 depósitos. El dispositivo es manejado a través de los siguientes registros:

Registro de Control (puerto 0x30):

Dir.	bit 7	bit 6	bit 5	bit 4	bit 3	bit 2	bit 1	bit 0
0x30	***B1***	***B0***	***ACTIVAR_B***	***X***	***X***	***S1***	***S0***	***LEER_S****

- **LEER_S***: Solicita la lectura (0) del sensor de agua indicado en los bits S1-S0.
- **S1-S0**: Sensor de agua a leer.
- **B1-B0**: Bomba a activar.
- **ACTIVAR_B**: Activa (1) o desactiva (0) la bomba de agua indicada en los bits B1-B0.

Registro de Estado (puerto 0x31):

Dir.	bit 7	bit 6	bit 5	bit 4	bit 3	bit 2	bit 1	bit 0
0x31	***NIVEL***	***X***	***X***	***X***	***X***	***LEÍDO****	***X***	***X***

- **LEÍDO***: Se activa (0) cuando finaliza la petición de lectura.
- **NIVEL**: Una vez finalizada la petición de lectura del sensor, indicará con 1 si el último estado fue lleno, y 0 si el último estado fue vacío.

a) Usando E/S por polling, realice en C una función que reciba como parámetro el depósito (posibles valores: 0, 1, 2, 3) del que se quiere leer el sensor y devuelva 1 si el último estado fue lleno y 0 en caso contrario.

b) Utilizando E/S por polling realice en C la función principal que verifique continuamente si algún depósito está vacío usando la función anterior. En dicho caso, debe activar la bomba de dicho depósito y esperar hasta que se detecte lleno. En el momento que se detecte lleno, debe apagar la bomba y continuar con el control del resto de depósitos.

Ejercicio 15. Dispone de un microcontrolador que gobierna un sistema de recolección de datos proveniente de varios sensores. Consta de tres sensores: presión, inclinación y altitud. El microcontrolador posee un controlador de entrada/salida separada con las siguientes características:

Registro de datos (puerto 0x30)

Registro de control (puerto 0x31):

Dir.	bit 7	bit 6	bit 5	bit 4	bit 3	bit 2	bit 1	bit 0
0x31	***ON***	***INT****	***S1***	***S0***	***LEER_S****	***X***	***X***	***X****

- **ON**: poner a 1 para encender el dispositivo
- **INT***: poner a 0 para activar modo por interrupciones, poner a 1 para activar modo polling
- **S1-S0**: indicar sensor a leer. 0: presión; 1: inclinación; 2: altitud
- **LEER_S***: poner a 0 para solicitar lectura del sensor indicado previamente en los bits S1-S0.

Registro de Estado (puerto 0x32):

Dir.	bit 7	bit 6	bit 5	bit 4	bit 3	bit 2	bit 1	bit 0
0x32	***X***	***X***	***X***	***X***	***LEÍDO****	***ON***	***X***	***X***

- **LEÍDO***: Se activa (0) cuando finaliza la petición de lectura y hay un dato que leer en el registro de datos
- **ON**: vale 1 si el dispositivo está encendido y 0 en caso contrario

a) Implemente la función de configuración inicial del sistema. Dicha función recibe como parámetro un valor entero que indica si se configura en modo polling (1) o en modo por interrupciones. En caso de habilitación por interrupciones, se debe registrar con el vector de interrupción 0xA. Antes de empezar con la configuración, debe comprobar si está encendido, en caso contrario, debe encenderlo y esperar a que lo esté.

b) Mediante polling, realice una función que reciba como parámetro el sensor que se desea leer (posibles valores: 0, 1, 2) y devuelva el valor de dicho sensor. Para ello, debe establecer en los bits S1-S0 el sensor que se desea leer, activar LEER_S*, esperar la activación de LEÍDO* y leer el dato del registro de datos.

Ejercicio 16. Se quiere desarrollar un sistema para la medición de monóxido de carbono (CO) y activación de alarma en el caso de alcanzar un valor perjudicial para la salud. Para ello, dispone de un dispositivo, que consta de 4 sensores de CO (8 bits de resolución) y 4 sirenas, conectado a un microcontrolador de bajo coste. El dispositivo es capaz de emitir interrupciones para detectar un nivel de CO nocivo (ALARM) y para indicar la finalización de una operación de lectura. El sistema posee un módulo de entrada/salida separada con las siguientes características:

Registro de control (puerto 0x30):

Dir.	bit 7	bit 6	bit 5	bit 4	bit 3	bit 2	bit 1	bit 0
0x30	***ON***	***INT****	***C_ALARM****	***SIRENA***	***S1***	***S0***	***LEER_S****	***X***

- ***ON:*** poner a 1 para encender el dispositivo.
- ***INT*:*** poner a 0 para activar modo por interrupciones, poner a 1 para activar modo polling.
- ***LEER_S**:** Solicita la lectura (0) del sensor de nivel de CO indicado en S1-S0.
- **S1-S0**: Indica sensor a leer.
- **C_ALARM**: Configura la activación automática de la alarma (1).
- **SIRENA**: Activa (1) la sirena de la habitación indicada por los bits S1 y S0.

Registro de Estado (puerto 0x31):

Dir.	bit 7	bit 6	bit 5	bit 4	bit 3	bit 2	bit 1	bit 0
0x31	***ALARM***	***S_ALARM1***	***S_ALARM0***	***X***	***X***	***LEÍDO****	**DESACT***	***LISTO***

- **DESACT***: Se activa (0) cuando el operario pulsa el botón para desactivar la alarma.
- **LEÍDO***: Se activa (0) cuando finaliza la petición de lectura. En ese momento, está disponible el valor medido de CO en el registro de datos.
- **LISTO**: vale 1 si el dispositivo está encendido y 0 en caso contrario. Tras leer el registro, se desactivan.
- **ALARM:** Notifica (1) que ha saltado la alarma por exceso de CO en la habitación indicada por ***S_ALARM1*** y ***S_ALARM0.***

Registro de Datos (puerto 0x32)

a) Implemente la función de configuración inicial del sistema. Dicha función recibe como parámetro un valor entero que indica si se configura en modo polling (1) o en modo por interrupciones (0). En caso de habilitación por interrupciones, se debe registrar con el vector de interrupción 0xA. Antes de empezar con la configuración, debe comprobar si está encendido, en caso contrario, debe encenderlo y esperar a que lo esté listo.

b) Considerando el dispositivo configurado, realice en C mediante E/S por polling una función que reciba por parámetros el número del sensor cuyo valor de CO queremos leer y lo devuelva.

c) Implemente la rutina de tratamiento de la interrupción que active la sirena cuando se active ***ALARM*** del registro de estado. La rutina sólo atenderá la activación de la alarma. Cuando salte la alarma deberá activar la sirena vinculada al sensor que ha provocado la alarma.

Ejercicio 17. Disponemos de un sistema de control automático de una silla de ruedas en base al movimiento de la cabeza de un usuario tetrapléjico. Para ello, realizaremos el control del sistema a través de dos registros de E/S comunicados con el controlador de la silla a través de E/S separada:

Mov. cabeza	**7**	**6**	**5**	**4**	**3**	**2**	**1**	**0**
Dir. 0xA	*Lee*	*Disponible*	*X*	*X*	*Mov. Lateral*		*Mov. Inclinación*	

- (Datos) Mov. Lateral: Dato del movimiento lateral (derecha e izquierda) de la cabeza
- (Datos) Mov. Inclinación: Dato del movimiento de inclinación (adelante y atrás) de la cabeza.
- (Control) Lee: activo en alta. Activar para solicitar lectura de datos (tanto *Lateral* como *Inclinación*).
- (Estado) Disponible: activo en alta. Aviso de dato disponible para lectura.

Motores Silla	**7**	**6**	**5**	**4**	**3**	**2**	**1**	**0**
Dir. 0xB	*COMANDO*						*Ocupado*	*Activar**

- (Estado) Ocupado: activo en alta. Aviso de motores ocupados.
- (Control) Activar*: activo en baja. Activar para solicitar ejecución de comando.
- (Datos) Comando: Dato que proporciona la orden de movimiento del motor de la silla de ruedas.

Se pide realizar un programa que, mediante E/S por polling, lea continuamente los valores de los movimientos *lateral* e *inclinación* del usuario y active el comando oportuno en el motor de la silla. Para ello, tenga en cuenta:

- Para leer los datos del usuario, se debe solicitar la lectura de los mismos mediante la señal de control correspondiente y esperar a que estén disponibles (señal de estado) antes de leer ambos.
- Para obtener el comando a aportar a los motores, una vez tenemos movimiento lateral e inclinación, debe usar la función ya implementada:

unsigned char comando (unsigned char lateral, unsigned char inclinacion)

- Para programar un movimiento en los motores de la silla solo podrá hacerlo si el controlador no está ocupado; y, una vez colocado el comando (obtenido de la función anterior), deberá activarlo mediante la señal de control correspondiente.

Ejercicio 18. Queremos programar el funcionamiento de los botones de un controlador de persiana eléctrica mediante E/S por interrupciones. Para ello, disponemos del siguiente registro de E/S ligado al controlador de la persiana:

Persiana	7	6	5	4	3	2	1	0
Dir. 0x05	*X*	*Pul_Arriba*	*Pul_Abajo*	*X*	*Fin_Arriba**	*Fin_Abajo**	*Mov_Arriba*	*Mov_Abajo*

- (Estado) Pul_Arriba: activo en alta. Avisa que el usuario ha accionado el pulsador de recogida de persiana.
- (Estado) Pul_Abajo: activo en alta. Avisa que el usuario ha accionado el pulsador de extensión de persiana.
- (Estado) Fin_Arriba*: activo en baja. Avisa si la persiana ha llegado a su punto máximo de recogida.
- (Estado) Fin_Abajo*: activo en baja. Avisa si la persiana ha llegado a su punto máximo de extensión.
- (Control) Mov_Arriba: activo en alta. 1: iniciar el movimiento de recogida de persiana, 0: parar movimiento. También sirve para conocer si la persiana está en movimiento de recogida.
- (Control) Mov_Abajo: activo en alta. 1: iniciar el movimiento de extensión de persiana, 0: parar movimiento. También sirve para conocer si la persiana está en movimiento de extensión.

El controlador provocará una interrupción si se activa cualquiera de sus cuatro bits de estado. Se actuará de la siguiente forma en base al evento recibido:

- Si el usuario ha pulsado el botón de recogida/extensión de persiana, deberemos activar el movimiento de recogida/extensión siempre y cuando no se encuentre la persiana en su posición máxima de recogida/extensión.
- Si la persiana llega al punto máximo de recogida/extensión, deberemos parar el movimiento de recogida/extensión.
- Si la persiana se encuentra en movimiento y el usuario realiza una pulsación en el mismo sentido que el movimiento actual (recogida o extensión), se parará la persiana en la posición en la que se encuentre.
- La parada de la persiana cuando llega a alguno de sus puntos máximos es prioritaria frente al resto de eventos.

Se pide programar la rutina de tratamiento de la interrupción (o rutina de servicio) para atender cualquiera de las fuentes de interrupción descritas, implementando el comportamiento indicado anteriormente.

Ejercicio 19. Como ingeniero del ejército de aviación, le encargan diseñar un sistema de actuación automática para posibles ataques sobre los aviones de última generación. Para ello, dispone de un conjunto de sensores que detectan automáticamente todos los peligros cercanos al avión y a cuyas señales de estado puede acceder a través del siguiente registro:

	7	6	5	4	3	2	1	0
Dir. 0x10	*MotorIz*	*MotorDe*	*Fuselaje**	*X*	*PresenciaIz*	*PresenciaDe*	*PresenciaTr*	*X*

- Fuselaje*: activo en baja. Se activa si ha habido alguna perforación en el fuselaje.
- PresenciaIz: activo en alta. Proyectil detectado directo al avión en la cercanía del ala izquierda.
- PresenciaDe: activo en alta. Proyectil detectado directo al avión en la cercanía del ala derecha.
- PresenciaTr: activo en alta. Proyectil detectado directo al avión en la cercanía de la cola.
- MotorIz: activo en alta. Fallo del motor izquierdo.
- MotorDe: activo en alta. Fallo del motor derecho.

De igual forma, para intervenir sobre el sistema de actuación, puede hacerlo a través del siguiente registro:

	7	6 5	4 3	2	1	0
Dir. 0x11	*Expulsión**	*Potencia MotorIz*	*Potencia MotorDe*	*Contramedidas**	*AlerónIz*	*AlerónDe*

- Expulsión*: activo en baja. Activar para expulsión inmediata del piloto.
- Potencia MotorIz: lectura/escritura. Consultar potencia actual del motor (de 0 a 3). También permite modificarla.
- Potencia MotorDe: lectura/escritura. Consultar potencia actual del motor (de 0 a 3). También permite modificarla.
- Contramedidas*: activar para lanzar contramedidas.
- AlerónIz: activo en alta. Activar para desplegar el flap del ala izquierda.
- AlerónDe: activo en alta. Activar para desplegar el flap del ala derecha.

Nota: Todas las señales de control y estado, una vez se realiza la acción vuelven automáticamente a su valor inicial.

La activación del sistema de actuación automática se realizará por interrupciones en base a los estados de peligro detectados. Se actuará de la siguiente forma en base al evento recibido:

1. Si detecta alguna perforación del fuselaje, el sistema debe expulsar automáticamente al piloto.
2. Si detecta un fallo en uno de los motores, deberá aumentar su potencia (en 1); salvo si su potencia actual es cero, en cuyo caso debe expulsar automáticamente al piloto.
3. Si detecta la cercanía de un proyectil en la parte trasera, deberá activar las contramedidas.
4. Si detecta la cercanía de un proyectil a alguna de las alas, debe aumentar la potencia del motor opuesto (al máximo) y activar el flap de dicha ala (con el fin de realizar una acrobacia para evitar el proyectil).

Se pide programar la rutina de tratamiento de la interrupción para atender cualquiera de las fuentes de interrupción descritas, implementando el comportamiento indicado anteriormente.

Ejercicio 20. Se quiere programar un microcontrolador para el control del motor de un sillón articulado mediante E/S separada. El controlador de E/S del motor dispone de los siguientes registros:

	7	6	5	4	3	2	1	0
Dir. 0x0A	*Enable*	*Ready*	*Int**	*X*	*Bot_Subida**	*Bot_Bajada**	*Mov_Arriba*	*Mov_Abajo*
Dir. 0X0B	*Posición en Grados*							

- (Control) Enable: Activar para habilitar el dispositivo
- (Estado) Ready: Indica que el dispositivo está listo para funcionar.
- (Control) Int*: Configura el dispositivo para actuar por interrupciones (0).
- (Estado) Bot_Subida*: Avisa que el usuario ha accionado el botón de subida (0).
- (Estado) Bot_Bajada*: Avisa que el usuario ha accionado el botón de bajada (0).
- (Control) Mov_Arriba: Activar para iniciar el movimiento de subida y desactivar para parar dicho movimiento. También sirve para conocer si el sillón está en movimiento de subida.
- (Control) Mov_Abajo: Activar para iniciar el movimiento de bajada y desactivar para parar dicho movimiento. También sirve para conocer si el sillón está en movimiento de bajada.
- (Dato) Posición en Grados: Contiene la posición en que se encuentra el motor. Rango [0, 45] grados.

a) Realice la función de configuración para el funcionamiento por interrupciones. Para configurar el dispositivo, previamente se debe comprobar si está listo, y en caso contrario, se debe habilitar y esperar a que esté listo. Para configurar el computador, se debe establecer la rutina de servicio de interrupción y habilitar la interrupción 0x7. Si se produce error devolverá 0, en caso contrario 1.

b) Realice la rutina de servicio de la interrupción para atender a la interrupción que provoca el controlador si se pulsa alguno de los botones. Se debe actuar de la siguiente forma:

I. Si el sillón está parado, deberemos activar el movimiento correspondiente al botón pulsado, siempre y cuando no se encuentre el sillón en su posición máxima de subida/bajada (0 y 45 grados respectivamente). La posición está disponible en el segundo registro del controlador.

II. Si el sillón se encuentra en movimiento, se debe para el movimiento.

III. No es necesario que implemente la parada del motor al llegar a su posición máxima de subida/bajada porque el dispositivo la realiza de forma automática.

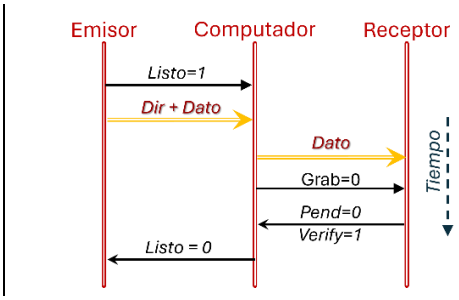
Ejercicio 21. Sea un computador conectado a dos dispositivos, uno Emisor y otro Receptor. El computador recibirá del dispositivo Emisor una dirección y un dato, y el computador deberá reenviar el dato al registro del dispositivo Receptor cuya dirección coincide con la proporcionada por el Emisor. Por ejemplo, si recibe del Emisor la dirección 4, enviará el dato recibido al registro de datos 4 del Receptor. El protocolo de comunicación para transferir un dato es el siguiente:

Dir base 100 **Dispositivo Emisor**

+0	dir2	dir1	dir0	dat3	dat2	*Listo*	dat1	dat0	*Reg.Común*

Dir base 200 **Dispositivo Receptor**

+0	1	1	1	1	dat3	dat2	dat1	dat0	*Reg.Datos 0*
	…								
+7	1	1	1	1	dat3	dat2	dat1	dat0	*Reg.Datos 7*
+8	x	*Pend*	*Grab*	x	x	*x*	*Verify*	x	*Reg.Común*

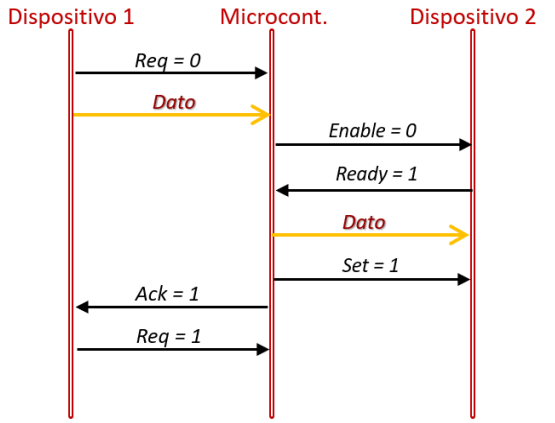


Tenga en cuenta que el *Receptor* confirma la grabación del Dato estableciendo *Pend*=0 y *Verify*=1 simultáneamente. Implemente en el *Computador* un programa en C mediante E/S separada que retransmita continuamente los datos recibidos del emisor al receptor según el cronograma anterior.

Ejercicio 22. Sea un sistema compuesto por dos dispositivos conectados a un microcontrolador, que se quiere programar para recibir datos del dispositivo 1 y trasmitirlo al dispositivo 2 según los siguientes criterios:

- **si el dato recibido es par, se debe transmitir el valor recibido sin modificar**
- **si el dato recibido es impar, se debe transmitir el valor recibido incrementado en 1.**

Se define el comportamiento detallado en el siguiente cronograma, y en la descripción de los registros de los dispositivos:



Registro Dispositivo 1 (0x19):

7	6	5	4	3	2	1	0
*Req**	*Ack*	*D3*	*D2*	*D1*	*D0*	*X*	*X*

Req*: Indica que hay un nuevo dato disponible en D3-D0.
Ack: Notifica al dispositivo que el dato ha sido procesado.
D3-D0: Dato.

Registro Dispositivo 2 (0x2F):

7	6	5	4	3	2	1	0
*Enable**	*Ready*	*D4*	*D3*	*D2*	*D1*	*D0*	*Set*

Enable*: Activar para habilitar el dispositivo.
Ready: Indica que el dispositivo está listo para recibir.
D4-D0: Dato.
Set: Notifica al dispositivo que se ha enviado un comando.

Se pide realizar un programa que, mediante E/S por polling, realice de forma repetitiva: la recepción del dispositivo 1 y transmisión del dato al dispositivo 2, modificado o no según se indica en el párrafo anterior (resaltado en negrita).

Ejercicio 23. Poseemos un controlador de E/S que se encuentra conectado a un dispositivo meteorológico, que controla presión y humedad del ambiente. El controlador de E/S posee tres registros a los que poder acceder, cada uno de ellos de un byte de tamaño, situados en las posiciones 0x30 (control), 0x31 (estado) y 0x32 (datos) del mapa de E/S del sistema (E/S aislada en nuestro computador). El dispositivo meteorológico será interrogado e irá proporcionando la información correspondiente: nos puede proporcionar la información utilizando datos de simple precisión (32 bits) o de doble (64 bits), dependiendo de la configuración que le aportemos al inicio. Aportará información byte a byte (registro de datos de un byte) y tendremos, a posteriori, que ir unificando estas informaciones.

Registro de control:

Modo[1]	Modo[0]	X	RESET*	REQ*	PRECISION	VEL[1]	VEL[0]

- **Modo [1-0]:** indica el modo de funcionamiento, pudiendo ser modo por petición (00), por ráfaga (01) o por interrupciones periódicas (10). El modo por petición solo nos proporcionará un dato cuando se lo indiquemos mediante la señal REQ*. El modo ráfaga irá proporcionándonos datos con una cierta cadencia (utilizado para no ir haciendo peticiones en el modo E/S por polling). En el último modo el controlador se configura con interrupciones: al activarlas, la interrupción queda asociada a la activación de la señal DATA* del registro de estado, que se activa cada vez que hay un byte disponible (ver descripción posterior), y a la activación de la señal ERROR*.
- **REQ*:** Petición de dato. En modo "por petición", habrá que activarlo para recibir cada uno de los bytes; en modo "por interrupciones", no tiene uso, pues el dispositivo, una vez encendido, comenzará a colocar datos e indicarlo en DATA* (registro de estado).
- **RESET***: Enciende el dispositivo si no lo estuviese o lo resetea en caso de error.
- **PRECISION:** indica si los datos aportados van a ser de 32 bits (0) o de 64 (1).
- **VEL [1-0]:** Velocidad de la ráfaga en el modo "01". En otro modo no es utilizado.

Registro de estado:

X	X	X	DATA*	ACK*	ON	BUSY*	ERROR*

- **DATA*:** el periférico activa esta señal cuando ha colocado un byte disponible en el registro de datos. Posteriormente, podremos leerlo. Se activará tras poner cada byte (no únicamente con el primer byte de cada palabra).
- **ACK*:** confirmación por parte del periférico de recepción de una petición de dato (solo usado en modo "por petición"). La comunicación bajo demanda funciona entonces de la siguiente forma: el usuario activa REQ* en el registro de control, el periférico activa ACK* en el registro de estado al recibir la petición, el periférico coloca el dato en el registro de datos y, por último, activa la señal de DATA* para indicar que el dato está disponible. Esta comunicación se usará en el control bajo demanda, únicamente para el primer byte (los restantes los irá proporcionando automáticamente tras cada lectura, activando

en cada caso la señal de DATA*). En modo "por interrupciones" esta comunicación entre REQ* y ACK* no es necesaria.

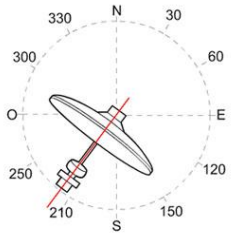
- **ON:** indica si está encendido.
- **BUSY*:** dispositivo ocupado.
- **ERROR*:** error de dispositivo. En ese caso resetearlo.

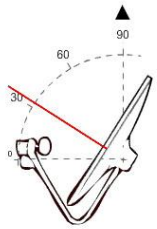
A la hora de solicitar los datos, el dispositivo nos devuelve en primera instancia la temperatura y, cuando termina con ella, comienza el envío de la humedad; así pues, no es necesario solicitar cada uno por separado. Se pide:

a) Realizar un control mediante E/S por polling para ir recibiendo, de forma periódica información en tiempo real del estado de los sensores (modo 00), utilizando para ello datos de simple precisión. Recuerde tener en cuenta el mecanismo de comunicación asíncrono que se ha explicado mediante el uso de REQ*, ACK* y DATA*. Tras recoger los datos, se almacenarán en un vector las temperaturas y en otro las humedades. Estos vectores tendrán un tamaño máximo de 1024 posiciones y, en caso de llenarse, se comenzará a escribir nuevamente por el principio.

b) Realizar un control por interrupciones (modo 10) del dispositivo creando, para ello, la rutina de inicialización (suponemos que se le asigna el vector de interrupciones 0xFF) y la rutina de interrupción del mismo. Esta rutina, tal como se comentó anteriormente, está asociada a la activación de DATA* y ERROR*. Así pues, deberá discriminar la fuente de interrupción para cada caso. En caso de error, se reinicia el dispositivo y, en caso de dato, se leerá el byte correspondiente (recuerde que una lectura de temperatura y humedad conlleva, al menos, 8 bytes; y en la rutina de interrupción leerá únicamente uno).

Ejercicio 24. Se desea controlar el giro de una antena parabólica mediante un computador. La antena dispone de 2 motores encargados de orientarla. El motor H controla el azimut (componente horizontal) y el motor V la Elevación (componente vertical).





Azimut (motor H) *Elevación (motor V)*

Tales motores están controlados por un dispositivo que se encuentra conectado al computador. El computador se comunica con el dispositivo mediante E/S separada (aislada) y sólo puede utilizar E/S por polling (sondeo). El dispositivo contiene los siguientes registros:

Registro de control de 8 bits (dirección de E/S: 0x350)

Bit	7	6	5	4	3	2	1	0
Función	Dirección Motor H	Habilita Motor H	Home	Dirección Motor V	Habilita Motor V	Siempre a 0		

- **Habilita Motor H o V:** con '1' el motor correspondiente gira en la dirección especificada por *Dirección Motor*. Con '0' el motor se detiene.
- **Dirección Motor V:** con '1' el giro del motor V hace aumentar el ángulo de elevación y con '0' lo contrario.
- **Dirección Motor H:** con '1' el giro del motor H hace rotar la antena en el sentido de las agujas del reloj (aumenta los grados del azimut) con '0' lo contrario.
- **Home:** Mueve los motores H y V hasta situarse en la posición inicial, esto es, elevación y azimut a 0 grados.

Registro de estado de 16 bits (dirección de E/S: 0x351)

Bit	15 14 13 12 11 10 9 8 7	6 5 4 3 2 1 0
Función	Azimut actual en grados (valores posibles: de 0 a 359)	Elevación actual en grados (valores posibles: de 0 a 90)

Para orientar la antena a un determinado satélite tendrá que activar cada motor y hacerlos girar en el sentido adecuado hasta alcanzar el satélite. Cuando alcance la elevación requerida deberá apagar el motor V inmediatamente, y de igual forma, deberá apagar el motor H cuando alcance el azimut requerido. Podrá verificar si ha alcanzado el ángulo requerido en ambos casos a través del registro de estado.

Nota: Recuerde que hay instrucciones de E/S para accesos de 8 bits (*inb* y *outb*) e instrucciones de E/S para accesos de 16 bits (*in* y *out*).

a) Realice una función en C que sitúe la antena en la posición inicial (Home). La función no debe terminar hasta que la antena no se encuentre situada en dicha posición.

void orienta_a_Home()

b) Realice una función en C que mueva el motor V hasta alcanzar la elevación en grados indicada como parámetro de la función (debe tener en cuenta la dirección hacia la que girar)

void orienta_Elevacion (unsigned char elevacion)

Ejercicio 25. Se dispone de un teclado con los siguientes registros de 8 bits:

Registro de datos: contiene el código de la tecla pulsada.

Registro de estado: el bit 7 indica si el teclado está ocupado (valor 0) o si hay un dato válido en el registro de datos (valor 1).

Registro de control: posee las siguientes características:

- El bit 0 debe ponerse a 1 para iniciar una lectura. La escritura en este registro pone a 0 el bit 7 del registro de estado.
- El resto de los bits se utilizan para configurar ciertas características del teclado que no son necesarias para este ejercicio.

Suponga que los tres registros siempre se encuentran en direcciones consecutivas (tanto si son direcciones de E/S separada como si están mapeadas en memoria), en este orden: registro de datos, registro de estado y registro de control. Se pide:

a) Escriba una función en C que lea un dato del teclado y lo devuelva como resultado de la función. Los registros se encuentran **mapeados en memoria** en la dirección indicada por el parámetro de la función. Su prototipo es el siguiente:

unsigned char lee_teclado(unsigned char *dir);

b) Escriba una función en C que lea un dato del teclado y lo devuelva como resultado de la función. Los registros se encuentran en el **espacio de direcciones de E/S** en la dirección indicada por el parámetro de la función. Su prototipo es el siguiente:

unsigned char lee_teclado(unsigned int puerto);

Ejercicio 26. Se pretende diseñar un sistema domótico. Para ello se dispone de un computador que se comunica con múltiples dispositivos de E/S mediante un protocolo propio. El sistema tiene la siguiente descripción:

Registro Datos *(accesible en la dirección REG_DATOS): entrada o salida dependiendo del comando utilizado.*

Registro Control *(accesible a través de la dirección REG_CONTROL):*

7	6	5	4	3	2	1	0
HAB_2	HAB_1	HAB_0	$TIPO_1$	$TIPO_0$	$ORDEN_1$	$ORDEN_0$	SEND

- HAB2-0: indicación numérica de la habitación sobre la que actuar (desde la 0 hasta la 7).
- TIPO1-0: tipo de elemento sobre el que actuar. En nuestro sistema:
  - 0: luces.
  - 1: persianas.
  - 2: aire acondicionado.
  - 3: sensor de temperatura.
- ORDEN1-0: comando a indicar al elemento seleccionado. En nuestro sistema:
  - 0: apagar (en el caso de las persianas, cerrar completamente).
  - 1 encender (en el caso de las persianas, abrir completamente).
  - 2: configurar valor.
    - Aire acondicionado: indicar antes los grados en el registro de datos.
  - 3: leer valor.
    - Sensor de temperatura: ver temperatura de habitación (devuelto por el reg. de datos).
    - Resto: comprobar si encendidos (1) o apagados (0), devuelto por el reg. de datos.
- SEND: activar para mandar el comando.

Registro Estado *(accesible a través de la dirección REG_ESTADO):*

7	6	5	4	3	2	1	0
X	X	X	X	X	X	ERROR	DATO

- ERROR: comando erróneo.
- DATO: dato devuelto en el registro de datos. Se ha de esperar cada vez que se solicita un dato.

a) Como medida de seguridad, se quiere implementar la utilidad del apagado de luces de toda la vivienda. Implemente una función que reciba por parámetros el número de habitaciones que posee la vivienda y, en base a ellas, las recorra y apague las luces (únicamente en aquellas en las que se quedaron encendidas). La función devolverá el número de habitaciones que tenían las luces encendidas.

b) Realice un control por polling de la temperatura de todas las habitaciones (las 7), optimizando el consumo eléctrico. Para ello, inicialice los aires acondicionados de todas las habitaciones de su vivienda a una temperatura de 26 grados. Su programa consultará de forma continua la temperatura de los sensores de cada habitación; y en aquellas en las que se alcance la temperatura objetivo, se apagarán las máquinas de aire acondicionado. Si la temperatura sube de los 28 grados, se volverán a encender.

Ejercicio 27. Tenemos un sistema antiincendios formado por 3 sensores de gas (ubicados en diferentes localizaciones del edificio) y por un actuador que habilita la salida de agua por los aspersores. Para su control, disponemos de los siguientes registros de E/S separada:

Sensores

	7	**6**	**5**	**4**	**3**	**2**	**1**	**0**
Dir. 0xA	*ON*	*Disponible*	*Error**	*Enciende*	*Configura*	*Sensor a leer/configurar*		*Lee**
Dir. 0xB	*Dato del sensor solicitado*							

- (Control) Sensor a leer/configurar: bits para indicar el sensor a leer (de 0 a 2); también utilizado en el proceso de configuración para indicar el sensor a activar (del 0 al 2).
- (Control) Configura: activo en alta. Una vez colocado el número del sensor, se activa para configurarlo/s.
- (Control): Lee*: activo en baja. Una vez indicado el sensor a leer, se activa para solicitar lectura de su valor.
- (Control): Enciende: activo en alta. Se activa para arrancar el dispositivo.
- (Estado) ON: activo en alta. 1: dispositivo encendido; 0: dispositivo apagado.
- (Estado) Disponible: activo en alta. Si activo (1), dato disponible para ser leído del registro de datos (Dir. 0xB).
- (Estado) Error*: activo en baja. Activo (0) si se ha producido un error durante la configuración.

Actuador

	7	**6**	**5**	**4**	**3**	**2**	**1**	**0**
Dir. 0xC	*Activar**	*ON*	*Potencia*					

- (Control) Activar*: activo en baja. Una vez colocado el valor de la potencia del actuador, activar para encenderlo.
- (Estado) ON: activo en alta. 1: dispositivo encendido; 0: dispositivo apagado.
- (Datos) Potencia: valor entero sin signo de 6 bits para indicar la potencia con la que activar el actuador.

Nota: Todas las señales de control y estado, una vez se realiza la acción pertinente vuelven automáticamente a su valor inicial.

a) Implemente la función de configuración del sistema, que recibe como parámetro el número del sensor a activar (de 0 a 2). *Proceso*: 1º) consultar si el dispositivo está apagado (en cuyo caso encenderlo y esperar a que lo esté); 2º) configurar el sensor recibido por parámetros; 3º) comprobar que no hay error (en cuyo caso finaliza y devuelve '1'); 4ª) comprobar que el actuador está encendido (en caso contrario, activarlo con potencia 0).

b) Implemente la función principal del sistema que, inicialmente, configure su dispositivo para que funcionen todos los sensores de humo y, posteriormente (de forma permanente), realice lecturas para cada uno de los sensores. En caso de que el nivel de gas de alguno supere 100, deberá activar el aspersor. La potencia de éste será proporcional al número de sensores de humo activos: potencia de 20 por cada sensor que supere el umbral.

Ejercicio 28. Se desea desarrollar una serie de rutinas para controlar un acelerómetro. El acelerómetro nos proporciona la aceleración en cada uno de los ejes 3 (x, y, z) con una resolución de 16 bits y además es capaz de detectar la caída libre (Free Fall). Para controlarlo dispone de los siguientes registros:

Registro de estado (0x4F8): Cualquier cambio en este registro (salvo señales Busy y Ready) provoca el disparo de la interrupción 0x1F (en caso de estar habilitada).

7	6	5	4	3	2	1	0
X	X	X	X	Ready	Busy	Data Ready	Free Fall

- **Ready:** indica que el acelerómetro está listo para funcionar.
- **Busy:** indica que el acelerómetro está ocupado realizando alguna acción.
- **Data Ready:** indica que hay datos listos para leer en los registros de datos con la información de las aceleraciones.
- **Free Fall:** indica que el acelerómetro está en caída libre cuando está habilitada su detección.

Registro de control (0x4F9): Utilizado para la configuración y la petición de lectura de la aceleración los ejes del acelerómetro.

7	6	5	4	3	2	1 0
X	X	Device En	IRQ En	Free Fall	Read Accel	Acel. Axis

- **Device Enable:** con el bit a 1 habilita el acelerómetro (se supone siempre encendido, así que solo se pone en modo "on line" una vez haya sido configurado).
- **IRQ Enable:** Habilita el disparo de la interrupción del acelerómetro.
- **Free Fall:** Con el bit a 1 habilita la detección de la caída libre.
- **Read Acceleration:** Con este bit a 1 se lanza una lectura de la aceleración.
- **Acceleration Axis:** Selecciona el eje del acelerómetro que desea leerse (0 – eje x, 1 - eje y, 2 – eje z), ha de usarse en combinación con *Read Acel*.

Registro de datos (0x4FA-0x4FB): En estos registros se almacenan los datos de la aceleración del eje seleccionado en el registro de control para su lectura. Los datos de estos registros sólo son válidos cuando el bit *Data Ready* vale 1. Tras su lectura, este bit pasa a valer 0 automáticamente.

7 6 5 4 3 2 1 0	7 6 5 4 3 2 1 0
Acel. Axis MSB	Acel. Axis LSB

a) Escribir las rutinas necesarias para inicializar, realizar una lectura secuencial de los tres ejes y detectar la caída libre, mediante entrada/salida por polling. Se deben de tener en cuenta los siguientes aspectos:
- Antes de escribir en el registro de control hay que consultar/esperar el bit de Busy.
- Configurar con Free Fall activado e IRQ desactivado.
- Tras habilitar el acelerómetro (Device Enable), hay que esperar a que el acelerómetro esté listo (bit de Ready).
- Para leer la aceleración de un eje se escribe en el registro de control el número del eje que se desea leer y el bit Read Accel., y esperar a que se active Data Ready.
- Los datos obtenidos deberán guardarse en las variables *int acel[3]* y *int freeFall*

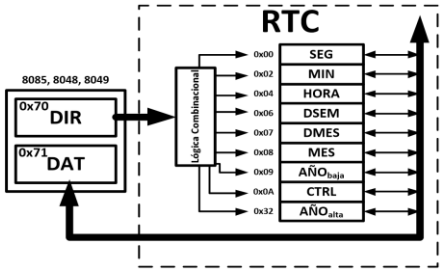
b) Realizar las mismas funciones que en el apartado anterior, pero habilitando y usando la interrupción del acelerómetro. Se debe de tener en cuenta que la interrupción del acelerómetro (con número de interrupción 0x1F) cada vez que se produce un cambio en el registro de estado (salvo señales Busy y Ready).

Ejercicio 29. Los computadores de la familia Intel disponen de un controlador de Reloj de Tiempo Real (RTC) que integran 256 registros internos. Algunos de estos registros contienen información del sistema como es el caso de la fecha y hora del sistema, la cual se actualiza gracias a un reloj con alarma que posee el propio controlador. Para acceder a los registros internos del dispositivo se utilizan los siguientes puertos:

- **Registro de dirección (0x70):** para especificar la dirección del registro interno al que se quiere acceder.
- **Registro de datos (0x71):** El acceso a este registro (tanto en lectura como escritura) provoca el acceso al registro interno que previamente se ha especificado en el registro de dirección (0x70).

Por ejemplo, para obtener el valor del registro interno 10, será necesario escribir un 10 en 0x70 y posteriormente leer el registro 0x71.

El siguiente diagrama muestra los registros internos relacionados con la fecha y hora:

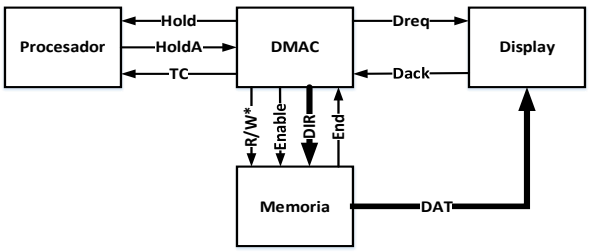


El reloj interno del controlador actualiza cada segundo los registros de la fecha y hora, lo que provoca que estos registros puedan tener valores inconsistentes durante el breve lapso que dura la actualización. Precisamente el bit UIP el cual corresponde al bit 7 del registro interno 0x0A, se encarga de informar cuando hay una actualización en curso (UIP = 1), por lo que el acceso a estos registros debe realizarse cuando UIP = 0.

Para conseguir una lectura sin errores, es necesario asegurarse que el bit UIP no vaya a pasar de '1' repentinamente durante el acceso a estos registros, para ello, es conveniente esperar el instante exacto en el que el bit pasa de 1 a 0 (esto es, detectar el flanco de bajada de UIP) pues a partir de este momento se dispone de algo menos de un segundo para acceder a los registros necesarios con seguridad.

Realice un programa que muestre por pantalla la fecha y hora actual utilizando E/S por polling.

Ejercicio 30. Se desea diseñar un DMAC empleando un microcontrolador. Este DMAC deberá comunicarse con un display externo, enviándole la información de los píxeles. El microcontrolador del DMAC dispone de un conjunto de registros para comunicarse con el display, con el procesador externo (para solicitar el acceso al bus) y con el controlador de memoria (para solicitar acceso a datos). El diagrama de bloques del sistema es el siguiente:



Nuestra tarea será programar el microcontrolador para que actúe como DMAC. Los registros que tiene son:

Nombre registro	Puerto
Estado/Control Procesador	0x26
Estado/Control Memoria	0x27
Dirección Memoria	0x28
Control Display	0x29
Estado Display	0x2A

7	6	5	4	3	2	1	0
X	X	X	HoldA	Hold	TC	X	X
X	End	R/W*	Enable	X	X	X	X
Dirección de acceso a memoria							
X	X	X	X	RST	X	X	Dreq
Dack	X	X	X	X	X	Err	X

La funcionalidad de cada uno de los bits especificados en los registros de control/estado es la siguiente:

- **Hold:** solicita al procesador el acceso al bus (activo en alta).
- **HoldA:** el procesador concede acceso al bus (activo en alta).
- **TC:** aviso al procesador de la finalización de la transferencia del bloque completo (activo en alta).
- **R/W*:** especifica si la operación a realizar sobre la memoria es de lectura (1) o escritura (0).
- **Enable:** comienzo de la operación de lectura/escritura sobre la memoria (activo en alta).
- **End:** memoria avisa al DMAC del fin de la operación de lectura/escritura (activo en alta). En caso de lectura, el dato se encuentra disponible en el bus de datos.
- **Dreq:** avisa al display que hay nuevo dato disponible en el bus de datos para ser leído (activo en alta).
- **Dack:** el display notifica al DMAC que ya ha procesado el dato disponible en el bus (activo en alta).
- **Err:** el display indica una situación anómala en su funcionamiento (activo en alta).
- **RST:** reinicia el display (activo en alta). Utilizado en caso de error.

El funcionamiento del DMAC es el siguiente:

1. Solicita al procesador el acceso al bus (Hold).
2. Espera la concesión del bus (HoldA).
3. Coloca la dirección del dato a acceder en el registro de dirección de memoria.
4. Activa la señal de lectura de memoria (R/W*) y habilita el acceso a la misma (Enable).
5. Espera a la activación del fin de operación de memoria (End).
6. Avisa al display que hay un dato disponible en el bus (Dreq).
7. Espera a que el display termine de procesar el dato (Dack).
8. Desactiva la señal de dato disponible (Dreq).
9. Espera a la desactivación de Dack.
10. ¿Hay más datos para enviar? Sí → Volver al paso 2.
    No → Desactiva la solicitud de acceso al bus (Hold).

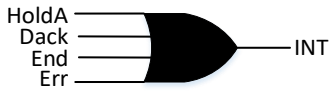
    Avisa al procesador que se ha completado la transferencia (TC).

A tener en cuenta:

- Suponga que la dirección base y el nº de bytes a transferir se encuentran en las variables *dirbase* y *numBytes*.
- Los accesos a memoria del DMAC se realizan sobre direcciones consecutivas y que es tarea del DMAC generar las direcciones de memoria a las que se accede.

a) Programe el código interno del DMAC para el control por polling del display. Para ello, implemente una función que lleve a cabo esta tarea, siguiendo los pasos indicados anteriormente.

b) Implemente la RSI para realizar un control por interrupciones, suponiendo que el DMAC debe controlar varios dispositivos externos y que la interrupción que recibe el DMAC es:


HoldA
Dack
End
Err
INT